

What is a Data Transfer Object (DTO)?

- A Data Transfer Object is a design pattern that allows for the exchange of data between software application subsystems or layers.
- Transfer data often between the business logic layer and the presentation or persistence layer.
- The primary goal of DTOs is to reduce the number of method calls between these layers by aggregating data into a single object.

Spring boot dto

- In Spring Boot application, DTOs are commonly used to encapsulate data transferred between the **controller and service** layers, or between the **service layer and the persistence layer**.
- It helps in maintaining a clear separation of concerns and improves the overall modularity of the codebase.

Benifits

1. Reduced Network Calls:

- DTOs help minimize the number of calls between different layers, especially in a microservices architecture where services communicate over a network.
- By bundling relevant data into DTOs, fewer network calls are required, resulting in improved performance.

2. Enhanced Maintainability:

- Separating the concerns of data transfer from the business logic enhances code maintainability.
- Changes in the underlying data structures or business logic don't necessarily impact the structure of DTOs, allowing for easier updates and modifications.

3. Versioning and Compatibility:

- DTOs can be versioned independently of the underlying business logic.
- This is particularly useful when dealing with evolving APIs or microservices, as it provides a level of backward compatibility by allowing older clients to still communicate using an older version of the DTO.

4. Reduced Overhead:

- Transmitting only the necessary data instead of entire entities reduces bandwidth and minimizes data transfer overhead. This is particularly relevant in scenarios where resources are constrained, such as mobile or low-bandwidth environments.

Dto in service layer

@Entity

```
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String username;  
    private String email;  
    // other fields, getters, and setters  
}
```

```
DTO UserDTO  
public class UserDTO {  
    private String username;  
    private String email;  
    // constructors, getters, and setters  
}
```

In the service layer, when retrieving or saving a user, convert between the entity and DTO:

@Service

```
public class UserService {  
    @Autowired  
    private UserRepository userRepository;  
    public UserDTO getUserById(Long userId) {  
        User user = userRepository.findById(userId).orElse(null);  
        return convertToDTO(user);  
    }  
  
    public void saveUser(UserDTO userDTO) {  
        User user = convertToEntity(userDTO);  
        userRepository.save(user);  
    }  
  
    private UserDTO convertToDTO(User user) {  
        // logic to convert User to UserDTO  
    }  
  
    private User convertToEntity(UserDTO userDTO) {  
        // logic to convert UserDTO to User  
    }  
}
```

- This way, the service layer communicates with the controller using DTOs, ensuring a clean separation between the data transfer and business logic.
-

Mapper (Converting Between Entity & DTO)

- **A Mapper is responsible for converting between Entity and DTO objects to maintain a clean architecture.**

```
@Component
```

```
public class UserMapper {  
    public UserDTO toDTO(UserEntity user) {  
        UserDTO dto = new UserDTO();  
        dto.setName(user.getName());  
        dto.setEmail(user.getEmail());  
        return dto;  
    }  
}
```

```
    public UserEntity toEntity(UserDTO dto) {  
        UserEntity user = new UserEntity();  
        user.setName(dto.getName());  
        user.setEmail(dto.getEmail());  
        return user;  
    }  
}
```

```
public class UserDTO {  
    private String name;  
    private String email;  
}
```

Service Layer (Business Logic Layer) with mapper

- The Service Layer contains business logic and calls the Repository for data manipulation.

@Service

```
public class UserService {  
    private final UserRepository userRepository;  
    private final UserMapper userMapper;  
  
    public UserService(UserRepository  
userRepository, UserMapper userMapper) {  
        this.userRepository = userRepository;  
        this.userMapper = userMapper;  
    }  
}
```

```
public UserDTO getUserById(Long id) {  
    UserEntity user = userRepository.findById(id)  
        .orElseThrow(() -> new RuntimeException("User not  
found"));  
    return userMapper.toDTO(user);  
}  
  
public void saveUser(UserDTO userDTO) {  
    UserEntity userEntity = userMapper.toEntity(userDTO);  
    userRepository.save(userEntity);  
}  
}
```

Controller (API Layer) with mapper

A Controller handles HTTP requests, calls the Service Layer, and returns responses.

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }
}
```

```
GetMapping("/{id}")
    public ResponseEntity<UserDTO>
    getUser(@PathVariable Long id) {
        return
        ResponseEntity.ok(userService.getUserById(id));
    }

@PostMapping
    public ResponseEntity<String>
    createUser(@RequestBody UserDTO userDTO) {
        userService.saveUser(userDTO);

        return
        ResponseEntity.status(HttpStatus.CREATED).body
        ("User created");
    }
}
```

DTO with layers

